Python Exception Handling

The Complete Placement & Interview Preparation Guide

TRY • EXCEPT • RAISE • CUSTOM EXCEPTIONS • INTERVIEW Q&A

From fundamentals to 45+ interview questions, coding programs, and a final revision cheat sheet

Table of Contents

1. Introduction — What is an Exception, Error vs Exception, Compile-time vs Runtime, Why Handling
2. Exception Handling Blocks — try, except, else, finally, execution flow
3. Common Built-in Exceptions — 9 exceptions with definition, example, output, interview explanation
4. The raise Keyword — working, examples, interview questions
5. Custom Exceptions — why, inheritance, pass, custom __init__
6. Assertions — assert, AssertionError, raise vs assert, best practices
7. Exception Hierarchy — BaseException, Exception, subclasses, diagram
8. Java vs Python — try/catch vs try/except, throw vs raise, checked vs unchecked
9. Coding Programs (12) — code, output, explanation
10. Placement Interview Questions (45) — one-line, 30-second, detailed answers
11. Quick Revision — cheat sheet, comparison tables, common mistakes, last-minute tips

1. Introduction

1.1 What is an Exception?

An **exception** is an event that disrupts the normal flow of a program's instructions during execution. When Python encounters a situation it cannot handle on its own — dividing by zero, opening a file that doesn't exist, accessing an index that doesn't exist — it creates an **exception object** and "raises" it. If nothing in the program catches this object, the program terminates and Python prints a traceback.

Exceptions are Python's way of saying: "*Something went wrong, and here is exactly what and where.*" Rather than letting a program crash silently or continue with corrupted data, Python raises a specific exception type so the developer can respond appropriately.

> **Interview angle:** Interviewers love to check whether you understand that exceptions in Python are **objects** — instances of classes that all ultimately derive from `BaseException`. This is different from languages where errors are just integer codes.

1.2 Error vs Exception

The words "error" and "exception" are often used interchangeably in casual conversation, but in precise technical terms they are different:

Aspect	Error	Exception
Definition	A serious problem that a reasonable application should usually not try to catch (e.g., `MemoryError`, `RecursionError`, syntax problems)	An issue that occurs during execution which *can* be caught and handled by the program
Recoverable?	Usually not recoverable	Usually recoverable via try/except
Python's model	`Error` is actually a subclass under `Exception` in Python (e.g., `ValueError`, `TypeError` are "Errors" that are also Exceptions)	Broad category — all runtime problems including Errors
Example	Syntax error, stack overflow	`ZeroDivisionError`, `KeyError`

In Python specifically, "Error" classes such as `ValueError` and `TypeError` are themselves subclasses of `Exception`. So in Python, nearly every "error" you catch is technically an "exception" too. The distinction matters more in general CS theory and in languages like Java.

1.3 Compile-time vs Runtime Errors

Type	When it occurs	Example	Can try/except catch it?
Compile-time (Syntax) Error	Before the program runs, while Python is parsing the code	Missing colon, unmatched bracket, incorrect indentation	No — the code never even starts running
Runtime Error (Exception)	While the program is executing	Dividing by zero, accessing a missing key	Yes — this is exactly what try/except is for

```
# Compile-time (SyntaxError) — this code will not even run
if True
    print("Missing colon above")

# Runtime (ZeroDivisionError) — code runs, then fails
x = 10 / 0
```

> **Interview angle:** A very common trick question is "Can you catch a SyntaxError with try/except?" The honest answer: generally no, because the interpreter fails before your try block is even reached. (The one exception: if you use `compile()` or `exec()` to dynamically run code, a resulting SyntaxError from that dynamic code CAN be caught, because the compilation itself happens at runtime.)

1.4 Why Exception Handling?

- **Prevents crashes:** Without handling, one bad input can terminate an entire application.
- **Graceful degradation:** Programs can log the problem and continue, or show a friendly message instead of a raw traceback.
- **Resource safety:** Using `finally` or context managers ensures files, network sockets, and database connections are always closed, even when something fails midway.
- **Separation of concerns:** Business logic doesn't need to be cluttered with endless "if valid" checks — errors are handled in one dedicated place.
- **Debuggability:** A well-raised exception carries a stack trace and message that pinpoints exactly where and why something failed.

> **Real-world framing for interviews:** "Exception handling lets a program fail safely and predictably instead of failing silently or crashing the entire application. It also lets us separate the 'happy path' logic from error-recovery logic, which keeps code readable."

2. Exception Handling Blocks

2.1 The `try` Block

Code that might raise an exception is placed inside a `try` block. Python attempts to execute it; if something goes wrong, control immediately jumps to a matching `except` block instead of crashing.

```
try:
    result = 10 / 0
```

2.2 The `except` Block

Catches and handles a specific exception type (or several). You can have multiple `except` blocks, catch multiple exception types in one block using a tuple, or catch everything generically.

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
except (TypeError, ValueError) as e:
    print(f"Type or value problem: {e}")
except Exception as e:
    print(f"Something else went wrong: {e}")
```

Interview angle: Order matters. Python checks `except` blocks top to bottom and stops at the first match, so always put more specific exceptions before more general ones (e.g., `ZeroDivisionError` before `Exception`).

2.3 The `else` Block

Runs only if the `try` block completes with **no exception**. It keeps the "happy path" code separate from the risky code in the `try` block, which is good practice.

```
try:
    result = 10 / 2
except ZeroDivisionError:
    print("Cannot divide by zero")
else:
    print(f"Success! Result is {result}")
```

```
Success! Result is 5.0
```

2.4 The `finally` Block

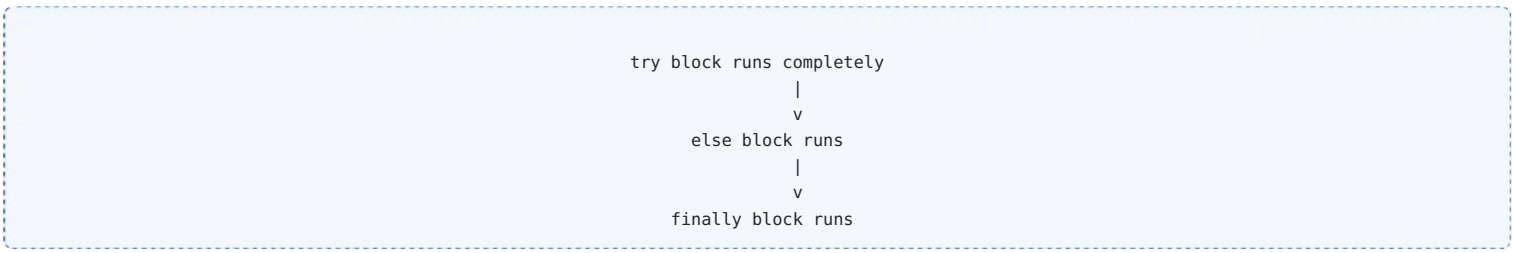
Always executes, whether an exception occurred or not — even if there is a `return`, `break`, or another exception inside the try/except. It is the standard place to release resources (close files, release locks, disconnect from a database).

```
try:
    f = open("data.txt")
    data = f.read()
except FileNotFoundError:
    print("File not found")
finally:
    print("Cleanup: closing resources")
```

```
File not found
Cleanup: closing resources
```

2.5 Execution Flow Diagrams

Case A: No exception occurs



Case B: Exception occurs and is caught

```
try block runs, error occurs
        |
        v
matching except block runs
        |
        v
(else is SKIPPED)
        |
        v
finally block runs
```

Case C: Exception occurs and is NOT caught

```
try block runs, error occurs
        |
        v
no matching except found
        |
        v
finally block STILL runs
        |
        v
exception re-raised upward,
program terminates (traceback)
```

Block	Runs when?	Mandatory?
`try`	Always attempted first	Yes (required to use except/finally)
`except`	Only if a matching exception is raised	At least one except or a finally must be present
`else`	Only if no exception occurred	Optional
`finally`	Always, no matter what	Optional

Classic interview question: "Does finally run even if there's a return statement in try or except?" — Yes. finally always runs, even overriding a pending return value if finally itself returns something (though doing that is considered bad practice).

3. Common Built-in Exceptions

3.1 ValueError

Definition: Raised when a function receives an argument of the correct type but an inappropriate value.

```
num = int("abc")
```

```
ValueError: invalid literal for int() with base 10: 'abc'
```

Interview explanation: ValueError signals "the type is fine, but this specific value doesn't make sense here" — e.g., converting a non-numeric string to int, or passing a negative number to something like `math.sqrt()` that expects it non-negative in real-number mode.

3.2 TypeError

Definition: Raised when an operation or function is applied to an object of an inappropriate type.

```
result = "5" + 5
```

```
TypeError: can only concatenate str (not "int") to str
```

Interview explanation: TypeError is about mismatched types — mixing str and int without conversion, calling a function with too many/few arguments, or trying to use something not-callable as if it were a function.

3.3 IndexError

Definition: Raised when a sequence (list, tuple, string) is accessed using an index that is out of range.

```
my_list = [1, 2, 3]
print(my_list[5])
```

```
IndexError: list index out of range
```

Interview explanation: IndexError is specific to positional/sequence access. It tells you the index itself is invalid for that sequence's current length — a very common bug in loop boundary conditions.

3.4 KeyError

Definition: Raised when a dictionary key is not found.

```
my_dict = {"name": "Alice"}
print(my_dict["age"])
```

```
KeyError: 'age'
```

Interview explanation: KeyError is the dictionary equivalent of IndexError. Interviewers often ask how to avoid it — using `.get(key, default)` or checking `if key in my_dict` instead of direct indexing.

3.5 NameError

Definition: Raised when a local or global name (variable or function) is not found — i.e., it was never defined or is out of scope.

```
print(undefined_variable)
```

```
NameError: name 'undefined_variable' is not defined
```

Interview explanation: NameError usually points to a typo, a variable used before assignment, or a scoping issue (e.g., referencing a variable defined only inside a function from outside it).

3.6 ZeroDivisionError

Definition: Raised when division or modulo operation has zero as the divisor.

```
result = 10 / 0
```

```
ZeroDivisionError: division by zero
```

Interview explanation: Purely arithmetic — applies to `/`, `//`, and `%`. Note that `10 / 0.0` also raises this in Python (unlike some languages that return infinity).

3.7 FileNotFoundError

Definition: A subclass of `OSError`, raised when a file or directory is requested but doesn't exist at the given path.

```
with open("nonexistent.txt") as f:
    data = f.read()
```

```
FileNotFoundError: [Errno 2] No such file or directory: 'nonexistent.txt'
```

Interview explanation: Common in file-handling questions. Interviewers may ask you to handle this gracefully with try/except rather than checking `os.path.exists()` first (checking first is prone to race conditions — "look before you leap" vs "easier to ask forgiveness than permission").

3.8 AttributeError

Definition: Raised when an attribute reference or assignment fails — i.e., the object doesn't have that attribute or method.

```
x = 5
x.append(10)
```

```
AttributeError: 'int' object has no attribute 'append'
```

Interview explanation: Very common when a function returns `None` unexpectedly and the caller then tries to call a method on it (e.g., a list method called on the `None` result of `list.sort()`, which returns `None`).

3.9 ImportError (and ModuleNotFoundError)

Definition: Raised when an import statement fails to find the module or name being imported. `ModuleNotFoundError` is a more specific subclass raised when the module itself can't be located.

```
import non_existent_module
```

```
ModuleNotFoundError: No module named 'non_existent_module'
```

Interview explanation: ImportError is the parent class; ModuleNotFoundError (added in Python 3.6) is thrown specifically when the module file itself is missing, whereas plain ImportError can also occur when the module exists but a specific name inside it can't be imported (e.g., `from math import nonexistent_func`).

Summary Table

Exception	Typical Trigger
ValueError	Right type, wrong value (e.g. int("abc"))
TypeError	Operation on incompatible types
IndexError	Sequence index out of range
KeyError	Dictionary key missing
NameError	Variable/name not defined
ZeroDivisionError	Division/modulo by zero
FileNotFoundError	File/path doesn't exist
AttributeError	Attribute/method doesn't exist on object
ImportError / ModuleNotFoundError	Module or name import fails

4. The `raise` Keyword

4.1 What is `raise`?

`raise` is used to manually trigger an exception — either a built-in one or a custom one you've defined. It lets a developer force an error to occur when business logic determines something is invalid, even if Python itself wouldn't have raised anything automatically.

```
raise ValueError("Age cannot be negative")
```

4.2 Why use `raise`?

- To enforce business rules (e.g., "withdrawal amount cannot exceed balance").
- To convert a low-level exception into a more meaningful, domain-specific one.
- To re-raise an exception after logging or partially handling it, so it still propagates.
- To fail fast on invalid input rather than let bad data silently flow further into the program.

4.3 Internal Working

When `raise` executes, Python:

1. Instantiates the exception object (if a class was given instead of an instance, Python calls it with no/given arguments to create one).
2. Attaches a traceback showing the call stack at the point of raising.
3. Halts normal execution and searches up the call stack for a matching `except` block.
4. If found, control transfers there; if not, the program terminates and Python prints the traceback to stderr.

4.4 Examples

```
# 1. Raise a built-in exception with a message
def set_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative")
    return age

# 2. Re-raise after logging
try:
    1 / 0
except ZeroDivisionError as e:
    print(f"Logging error: {e}")
    raise   # re-raises the SAME exception, preserving the traceback

# 3. Raise from another exception (exception chaining)
try:
    int("abc")
except ValueError as e:
    raise RuntimeError("Failed to process input") from e
```

```
ValueError: Age cannot be negative
Logging error: division by zero
Traceback (most recent call last):
  ...
ZeroDivisionError: division by zero

RuntimeError: Failed to process input
```

Bare `raise` vs `raise e`: A bare `raise` inside an except block re-raises the original exception with its original traceback intact. Writing `raise e` instead resets part of the traceback information, which is why the bare form is preferred when re-raising.

4.5 Interview Questions on `raise`

Q: What's the difference between an exception being "raised" and being "thrown"?
Python uses the term "raise"; Java/C++ use "throw." They describe the same mechanism — the language keyword differs by convention.

Q: Can you raise a string instead of an exception object in Python 3?
No. Python 3 requires raising an instance (or class) that derives from BaseException; raising a plain string raises a TypeError.

Q: What does "raise ... from ..." do?

It explicitly chains exceptions, setting the __cause__ attribute so the traceback shows both the original and new exception, clarifying that the second was triggered while handling the first.

5. Custom Exceptions

5.1 Why Create Custom Exceptions?

Built-in exceptions are generic. In a real application you often want errors that describe your *domain* precisely — e.g., `InsufficientFundsError` tells you far more than a generic `ValueError`. Custom exceptions:

- Make error handling code self-documenting.
- Let calling code catch your specific exception type without accidentally catching unrelated errors.
- Can carry extra data relevant to the failure (e.g., the account balance at the time of failure).

5.2 Why Inherit from `Exception`?

All custom exceptions should inherit, directly or indirectly, from Python's built-in `Exception` class (not from `BaseException` directly). This:

- Ensures your exception plugs into the standard hierarchy so generic `except Exception` blocks elsewhere in the codebase still catch it if needed.
- Gives you all the built-in machinery for free — `args`, string representation, traceback support.
- Avoids accidentally becoming a "system-exiting" exception, since things like `SystemExit` and `KeyboardInterrupt` inherit directly from `BaseException` and are deliberately excluded from broad `except Exception` catches.

```
class InsufficientFundsError(Exception):
    pass
```

5.3 Why Use `pass`?

When a custom exception doesn't need any special behavior beyond what `Exception` already gives it, the class body is left empty using `pass`. This is not lazy code — it's a deliberate, minimal "marker class" whose entire purpose is its distinct **type/name**, used purely so it can be raised and caught specifically.

```
class InvalidAgeError(Exception):
    pass

class NegativeDepositError(Exception):
    pass
```

5.4 What is a "New Exception Type"?

Every `class SomeError(Exception): pass` statement creates a genuinely new, distinct type in Python's type system. Even though it behaves identically to its parent, `isinstance()` and `except SomeError:` will only match this new class (and its own subclasses) — not sibling classes. This lets you build a whole family tree of related, catchable error types for one application.

5.5 Custom `__init__()` Examples

When you need to attach extra context (like an account balance, an invalid value, or an error code), override `__init__` and call `super().__init__()` to preserve normal exception behavior:

```
class InsufficientFundsError(Exception):
    def __init__(self, balance, amount):
        self.balance = balance
        self.amount = amount
        message = f"Cannot withdraw {amount}: balance is only {balance}"
        super().__init__(message)

try:
    balance = 500
    withdrawal = 800
    if withdrawal > balance:
        raise InsufficientFundsError(balance, withdrawal)
except InsufficientFundsError as e:
    print(f"Error: {e}")
    print(f"Shortfall: {e.amount - e.balance}")
```

```
Error: Cannot withdraw 800: balance is only 500
Shortfall: 300
```

Interview angle: Always call `super().__init__(message)` in a custom exception's `__init__`. Skipping it means `str(exception)` won't return your message properly, and tools that rely on `args` (like logging frameworks) may break.

6. Assertions

6.1 `assert`

The `assert` statement is a debugging aid that tests whether a condition is true. If the condition is false, it raises an `AssertionError`, optionally with a message.

```
def calculate_average(numbers):
    assert len(numbers) > 0, "List cannot be empty"
    return sum(numbers) / len(numbers)

calculate_average([])
```

```
AssertionError: List cannot be empty
```

6.2 `AssertionError`

This is the specific exception type raised by a failed `assert`. It can be caught like any other exception, though this is uncommon — assertions are meant to catch *programmer* mistakes (invariants that should never be false), not to validate normal, expected user input.

6.3 `raise` vs `assert`

Aspect	`assert`	`raise`
Purpose	Internal sanity check / debugging aid	Signal an actual application-level error condition
Exception type	Always AssertionError	Any exception type you choose
Can be disabled?	Yes — stripped out when Python runs with the `-O` (optimize) flag	No — always executes
Use for user input validation?	No — never rely on it for real validation	Yes — the correct tool for this

6.4 Best Practices

- Never use `assert` to validate untrusted user input or enforce security checks — it can be globally disabled with the `-O` flag, silently removing your checks.
- Use assertions to catch "this should be logically impossible" states while developing/testing.
- Use `raise` with specific exception types for anything the program must always check, in every environment.
- Always include a descriptive message in assertions — `assert x > 0, "x must be positive"` is far more useful when it fails than a bare `assert x > 0`.

Common trap question: "Can I use assert to check that a user's password meets the minimum length?" — No. Interviewers want to hear that assertions can be compiled away with `python -O`, making this a security hole; use an explicit `if ... raise ValueError(...)` instead.

7. Exception Hierarchy

7.1 `BaseException`

The root of Python's entire exception hierarchy. Every exception, built-in or custom, ultimately derives from it. Directly under it sit some special cases that are deliberately NOT meant to be caught by generic handlers: `SystemExit`, `KeyboardInterrupt`, and `GeneratorExit`.

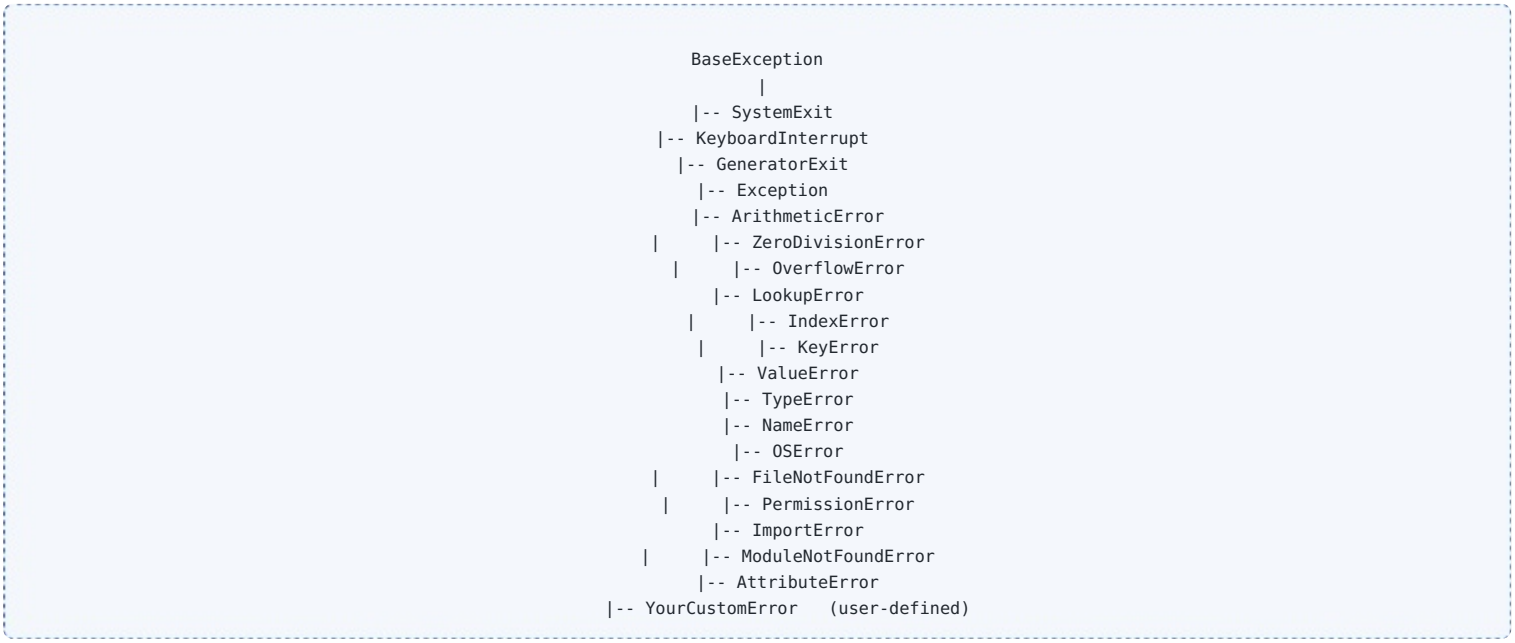
7.2 `Exception`

A direct subclass of `BaseException`, and the class that almost everything else — and every custom exception you write — should inherit from. When you write `except Exception:`, you catch essentially all "normal" runtime errors, while still letting a user's Ctrl+C (`KeyboardInterrupt`) pass through uncaught.

7.3 Common Subclasses

Class	Parent	Notes
ArithmeticError	Exception	Parent of ZeroDivisionError, OverflowError
ZeroDivisionError	ArithmeticError	Division/modulo by zero
LookupError	Exception	Parent of IndexError, KeyError
IndexError	LookupError	Sequence index out of range
KeyError	LookupError	Dict key missing
ValueError	Exception	Correct type, wrong value
TypeError	Exception	Wrong/incompatible type
NameError	Exception	Undefined name
OSError	Exception	Parent of FileNotFoundError, PermissionError
ImportError	Exception	Parent of ModuleNotFoundError
AttributeError	Exception	Missing attribute/method

7.4 Simple Hierarchy Diagram



Interview angle: "Why doesn't except Exception catch a Ctrl+C?" — Because KeyboardInterrupt inherits directly from BaseException, deliberately bypassing Exception, so that a broad error handler in a long-running program doesn't accidentally swallow the user's intentional interrupt signal.

8. Java vs Python Exception Handling

8.1 `try/catch` vs `try/except`

```
// Java
try {
    int result = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("Error: " + e.getMessage());
} finally {
    System.out.println("Cleanup");
}

# Python
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print(f"Error: {e}")
finally:
    print("Cleanup")
```

Structurally almost identical — the only real difference is the keyword (`catch` vs `except`) and Python's additional optional `else` block, which Java does not have.

8.2 `throw` vs `raise`

```
// Java
throw new IllegalArgumentException("Invalid input");

# Python
raise ValueError("Invalid input")
```

Functionally equivalent — both manually trigger an exception. Java requires the thrown object to be an instance of `Throwable`; Python requires an instance of `BaseException`.

8.3 Checked vs Unchecked Exceptions

This is the single biggest conceptual difference between the two languages.

- **Java** distinguishes *checked* exceptions (like `IOException`) — which the compiler forces you to either catch or declare with `throws` — from *unchecked* exceptions (like `NullPointerException`, subclasses of `RuntimeException`), which require no such declaration.
- **Python has no checked exceptions at all.** Every exception in Python behaves like Java's "unchecked" ones — nothing forces you to catch or declare anything. This is a deliberate design philosophy: Python trusts developers to handle errors where it makes sense, rather than enforcing it at compile time.

8.4 Comparison Table

Aspect	Java	Python
Keywords	try, catch, finally	try, except, else, finally
Manual trigger	throw	raise
Root class	Throwable	BaseException
Checked exceptions	Yes — enforced by compiler	No such concept
Declaring possible exceptions	Required via "throws" for checked exceptions	Not required at all
Multi-catch	`catch (IOException | SQLException e)`	`except (IOError, ValueError) as e:`
Custom exceptions	Extend Exception or RuntimeException	Extend Exception
"else" clause	Not available	Available — runs if no exception occurred
Resource cleanup	try-with-resources	finally, or "with" (context managers)

Interview angle: If asked "what's the biggest philosophical difference?" the strongest answer is: Java enforces exception-safety at compile time via checked exceptions; Python is fully dynamic and relies on runtime discipline and documentation instead — reflecting Python's broader "consenting adults" philosophy.

9. Coding Programs

Program 1: Simple Division with Exception Handling

```
def safe_divide(a, b):
    try:
        result = a / b
    except ZeroDivisionError:
        return "Error: Cannot divide by zero"
    else:
        return f"Result: {result}"
    finally:
        print("Division attempt finished")

print(safe_divide(10, 2))
print(safe_divide(10, 0))
```

```
Division attempt finished
Result: 5.0
Division attempt finished
Error: Cannot divide by zero
```

Explanation: Demonstrates all four blocks together. The first call succeeds so `else` runs; the second triggers `except`. `finally` runs in both cases.

Program 2: File Handling with Exception Handling

```
def read_file(path):
    try:
        with open(path, "r") as f:
            return f.read()
    except FileNotFoundError:
        return "Error: File does not exist"
    except PermissionError:
        return "Error: No permission to read this file"

print(read_file("missing.txt"))
```

```
Error: File does not exist
```

Explanation: Using `with` guarantees the file handle closes automatically even on exception, while distinct except blocks handle distinct real-world failure modes (missing file vs permission denied).

Program 3: ATM PIN Validation

```
class InvalidPinError(Exception):
    pass

def validate_pin(pin):
    if not pin.isdigit():
        raise InvalidPinError("PIN must contain digits only")
    if len(pin) != 4:
        raise InvalidPinError("PIN must be exactly 4 digits")
    return "PIN accepted"

try:
    print(validate_pin("12a4"))
except InvalidPinError as e:
    print(f"Validation failed: {e}")
```

```
Validation failed: PIN must contain digits only
```

Explanation: A custom exception type (`InvalidPinError`) makes the calling code's intent explicit — it's not a generic ValueError, it's specifically an ATM PIN problem.

Program 4: Bank Account Withdrawal

```
class InsufficientFundsError(Exception):
    def __init__(self, balance, amount):
        super().__init__(f"Cannot withdraw {amount}, balance is {balance}")
        self.balance = balance
        self.amount = amount
```

```
class BankAccount:
    def __init__(self, balance=0):
        self.balance = balance

    def withdraw(self, amount):
        if amount > self.balance:
            raise InsufficientFundsError(self.balance, amount)
        self.balance -= amount
        return self.balance

acc = BankAccount(1000)
try:
    acc.withdraw(1500)
except InsufficientFundsError as e:
    print(e)
```

```
Cannot withdraw 1500, balance is 1000
```

Explanation: Shows a custom exception with an overridden `__init__` carrying extra context (balance, requested amount) that the calling code could use for logging or a UI message.

Program 5: Login Validation

```
class AuthenticationError(Exception):
    pass

users = {"alice": "pass123"}

def login(username, password):
    if username not in users:
        raise AuthenticationError("Username not found")
    if users[username] != password:
        raise AuthenticationError("Incorrect password")
    return "Login successful"

for uname, pwd in [("alice", "wrong"), ("bob", "x"), ("alice", "pass123")]:
    try:
        print(login(uname, pwd))
    except AuthenticationError as e:
        print(f"Login failed for {uname}: {e}")
```

```
Login failed for alice: Incorrect password
Login failed for bob: Username not found
Login successful
```

Explanation: A single custom exception type covers multiple related failure reasons, distinguished by the message — a common, simple pattern for form/login validation.

Program 6: Custom Exception Hierarchy

```
class AppError(Exception):
    """Base class for all application-specific errors"""
    pass

class InvalidAgeError(AppError):
    pass

class InvalidEmailError(AppError):
    pass

def register_user(age, email):
    if age < 18:
        raise InvalidAgeError("User must be 18 or older")
    if "@" not in email:
        raise InvalidEmailError("Email must contain '@'")
    return "Registration successful"

try:
    register_user(15, "test@mail.com")
except AppError as e:
    print(f"Registration blocked: {type(e).__name__} - {e}")
```

```
Registration blocked: InvalidAgeError - User must be 18 or older
```

Explanation: Both specific exceptions inherit from a common `AppError` base, so a single `except AppError` block can catch any application-specific error while `type(e).__name__` still tells you exactly which one occurred.

Program 7: Assertion-Based Validation

```
def calculate_bmi(weight_kg, height_m):
    assert weight_kg > 0, "Weight must be positive"
    assert height_m > 0, "Height must be positive"
    return weight_kg / (height_m ** 2)

try:
    print(calculate_bmi(70, 0))
except AssertionError as e:
    print(f"Invalid input: {e}")
```

```
Invalid input: Height must be positive
```

Explanation: Assertions here act as internal sanity guards during development/testing; in production-facing validation you'd typically replace these with explicit `raise ValueError(...)` calls instead (see Section 6.4).

Program 8: Multiple Exceptions in One Program

```
def process(data, index, key):
    try:
        value = data[index]
        return value[key]
    except IndexError:
        return "Index out of range"
    except KeyError:
        return "Key not found"
    except TypeError:
        return "Type mismatch — check data structure"

records = [{"name": "Alice"}, {"name": "Bob"}]
print(process(records, 0, "name"))
print(process(records, 5, "name"))
print(process(records, 0, "age"))
```

```
Alice
Index out of range
Key not found
```

Explanation: One function, several possible failure points, each mapped to its own specific except block — a realistic pattern for defensive data-access code.

Program 9: Nested Try/Except

```
def parse_and_divide(a_str, b_str):
    try:
        a = int(a_str)
        try:
            b = int(b_str)
            return a / b
        except ZeroDivisionError:
            return "Inner error: division by zero"
    except ValueError:
        return "Outer error: invalid integer input"

print(parse_and_divide("10", "0"))
print(parse_and_divide("abc", "5"))
```

```
Inner error: division by zero
Outer error: invalid integer input
```

Explanation: The inner try/except only guards the division step; a ValueError from either conversion propagates outward to the outer except, illustrating how nested blocks scope their error handling narrowly.

Program 10: Exception Chaining with `raise ... from`

```
class ConfigError(Exception):
    pass

def load_config(value):
    try:
        return int(value)
    except ValueError as e:
        raise ConfigError(f"Invalid config value: {value}") from e
```

```
try:
    load_config("not_a_number")
except ConfigError as e:
    print(f"{e}")
    print(f"Caused by: {e.__cause__}")
```

```
Invalid config value: not_a_number
Caused by: invalid literal for int() with base 10: 'not_a_number'
```

Explanation: `raise ... from e` preserves the original low-level cause while surfacing a higher-level, domain-specific exception — useful in layered applications (e.g., config loading, API wrappers).

Program 11: Using `finally` for Resource Cleanup (Simulated Connection)

```
class Connection:
    def __init__(self, name):
        self.name = name
        self.open = True
        print(f"Opened connection: {name}")

    def close(self):
        self.open = False
        print(f"Closed connection: {self.name}")

def run_query(conn, query):
    try:
        if query == "":
            raise ValueError("Query cannot be empty")
        print(f"Running query on {conn.name}: {query}")
    finally:
        conn.close()

conn = Connection("DB1")
try:
    run_query(conn, "")
except ValueError as e:
    print(f"Query failed: {e}")
```

```
Opened connection: DB1
Closed connection: DB1
Query failed: Query cannot be empty
```

Explanation: Even though `run_query` raises before finishing, `finally` guarantees `conn.close()` executes — the exact reason resource cleanup belongs in finally (or a context manager) rather than after the risky code.

Program 12: Custom Exception with Multiple Attributes (Order Validation)

```
class OrderError(Exception):
    def __init__(self, message, order_id, error_code):
        super().__init__(message)
        self.order_id = order_id
        self.error_code = error_code

def place_order(order_id, quantity, stock):
    if quantity > stock:
        raise OrderError(
            f"Insufficient stock for order {order_id}",
            order_id=order_id,
            error_code="STOCK_001"
        )
    return "Order placed"

try:
    place_order("ORD-42", quantity=10, stock=3)
except OrderError as e:
    print(f"[{e.error_code}] Order {e.order_id} failed: {e}")
```

```
[STOCK_001] Order ORD-42 failed: Insufficient stock for order ORD-42
```

Explanation: Attaching structured attributes (`order_id`, `error_code`) to a custom exception lets calling code programmatically react to the failure (e.g., return a specific HTTP status or error code) rather than parsing the message string.

10. Placement Interview Questions (45)

1 What is an exception?

An event that disrupts normal program flow during execution.

An exception is an object Python creates when something goes wrong while a program is running — like dividing by zero or accessing a missing key — allowing the error to be caught and handled instead of crashing the program.

Exceptions represent runtime problems distinct from syntax errors. They are instances of classes derived from BaseException, carry a message and traceback, and can be caught with try/except to allow graceful recovery.

2 Why do we need exception handling?

To prevent crashes and handle errors gracefully.

Without handling, a single unexpected input or missing resource would terminate the whole program. Exception handling lets us catch these problems, respond appropriately (retry, log, show a message), and keep the application running.

It also improves code organization by separating "happy path" logic from error-recovery logic, ensures resources are released safely via finally/context managers, and produces clearer diagnostics for debugging.

3 Difference between Error and Exception?

Errors are usually unrecoverable; exceptions are usually recoverable.

In general CS terms, Errors are serious problems (like out-of-memory) you typically don't try to catch, while Exceptions are conditions you can catch and handle. In Python specifically, "Error" classes like ValueError are actually subclasses of Exception.

Python doesn't have a strict separate "Error" category the way some theoretical models do — most Python "errors" (ValueError, TypeError, KeyError) are exceptions you're expected to catch. True unrecoverable errors are rarer (e.g., MemoryError, RecursionError).

4 Explain try, except, else, and finally.

try = risky code, except = handle errors, else = runs if no error, finally = always runs.

try wraps code that might fail. except catches specific exception types. else runs only when try succeeds with no exception. finally always executes, used for cleanup regardless of outcome.

Execution order: try is attempted; on success, else runs, then finally. On exception, the first matching except runs (else is skipped), then finally still runs. If no except matches, finally still runs before the exception propagates upward.

5 What is the raise keyword?

Manually triggers an exception.

raise lets you deliberately signal that an error condition has occurred — either a built-in exception or a custom one — even when Python itself wouldn't automatically raise anything.

raise instantiates the exception (if a class was passed), attaches a traceback, and transfers control up the call stack looking for a matching except block. A bare raise inside an except block re-raises the currently-handled exception with its original traceback.

6 What is a custom exception?

A user-defined exception class, usually inheriting from Exception.

A custom exception is a class you write yourself, inheriting from Exception, to represent an error condition specific to your application's domain — e.g., InsufficientFundsError — rather than relying only on generic built-ins.

Custom exceptions improve code clarity and let calling code catch precisely the failure it cares about. They can be as simple as `class MyError(Exception): pass` or carry extra attributes via a custom __init__.

7 Why inherit from Exception rather than BaseException?

So it's caught by normal except Exception handlers, but not confused with system-exiting signals.

Exception is the branch of the hierarchy meant for "normal" catchable errors, while BaseException also covers SystemExit and KeyboardInterrupt, which should NOT be accidentally caught by broad error handlers.

Inheriting from Exception keeps your custom error compatible with common code patterns like `except Exception as e`, ensures it doesn't interfere with program termination signals, and follows the convention every standard library exception itself follows.

8 Why use pass in a custom exception's body?

Because the class needs no extra behavior — its name/type IS the point.

When a custom exception doesn't need special attributes or logic, `pass` is used because inheriting from Exception already provides all necessary functionality — the class exists purely to be a distinctly-named, catchable type.

This is a common and intentional Python idiom, not laziness. It creates a minimal "marker class" enabling precise try/except targeting without any additional overhead.

9 What is meant by a "new exception type"?

A distinct class in Python's type system, even if behaviorally identical to its parent.

Every class you define with `class X(Exception): pass` becomes a genuinely separate type. isinstance() checks and except clauses will match this new type specifically, not sibling exception classes, even though the behavior is inherited.

This lets developers build a hierarchy of related but distinguishable error types (e.g., AppError -> InvalidAgeError, InvalidEmailError), enabling both broad catching (except AppError) and narrow catching (except InvalidAgeError) as needed.

10 **What is assert used for?**

A debugging aid that raises AssertionError if a condition is false.

assert checks that a condition holds true; if it's false, it raises AssertionError with an optional message. It's meant to catch internal logic mistakes during development, not to validate untrusted user input.

Assertions can be globally disabled by running Python with the -O flag, which strips them out entirely — this is exactly why they should never be relied on for security or input validation in production code.

11 **Difference between raise and assert?**

raise always executes and can raise any exception; assert can be disabled and only raises AssertionError.

raise is for real application-level error signaling and always runs. assert is a lightweight sanity check meant for development/debugging, always raises AssertionError specifically, and can be stripped out entirely with the -O flag.

Use raise for anything that must be enforced in every environment, including production (input validation, business rules). Use assert only for conditions that should logically always be true and whose failure indicates a programming bug, not user error.

12 **Does finally always execute?**

Yes — even with return, break, or an unhandled exception.

finally executes no matter what happens in try or except — whether the code succeeded, raised a caught exception, raised an uncaught exception, or even hit a return statement.

The only practical exceptions are if the entire process is killed (os._exit(), a segfault, or the machine losing power) — normal control-flow statements like return/break/continue do not skip finally.

13 **Difference between Exception and BaseException?**

BaseException is the root of everything; Exception is the branch for normal, catchable errors.

BaseException is the top of the hierarchy and includes SystemExit and KeyboardInterrupt. Exception is a subclass of BaseException that covers essentially all standard runtime errors, and is what all custom exceptions should inherit from.

Using `except Exception` catches nearly everything you'd want to catch in normal application code while deliberately letting system-exit signals and keyboard interrupts propagate, preserving the ability to cleanly stop a program.

14 **What happens if an exception is not caught?**

The program terminates and prints a traceback.

If no except block matches the raised exception anywhere up the call stack, Python stops executing the program, prints the full traceback (call stack and error message) to standard error, and exits with a non-zero status.

Even though the program terminates, any pending finally blocks along the way still execute first, allowing partial cleanup before the process actually ends.

15 **Can you have multiple except blocks for one try?**

Yes, and Python checks them top to bottom, stopping at the first match.

You can stack multiple except blocks to handle different exception types differently. Python evaluates them in order and executes only the first one whose type matches, so specific exceptions should come before general ones.

You can also combine multiple types into one except using a tuple, e.g., `except (TypeError, ValueError) as e`. Placing a generic `except Exception` first would incorrectly swallow all more specific cases below it, so ordering matters.

16 **What is the purpose of the else clause in try/except?**

Runs code only if the try block succeeds with no exception.

else keeps success-path code separate from the risky code being tried, so that except blocks only catch errors from the specific operations meant to be guarded, not from the follow-up logic.

Without else, developers sometimes put success-handling code inside the try block itself, which risks accidentally catching exceptions raised by that follow-up code too — else avoids this by only running after try fully succeeds.

17 **Can a finally block suppress an exception?**

Yes, if it contains a return, break, or continue statement.

If finally itself contains a return, break, or continue, it can silently discard a pending exception from try/except — generally considered bad practice because it hides real errors.

This is a well-known gotcha: `finally: return value` overrides any exception that was propagating, which is why style guides recommend avoiding control-flow statements inside finally blocks.

18 **What is exception chaining (raise ... from ...)?**

Links a new exception to the original one that caused it.

`raise NewError(...) from original_exception` sets the __cause__ attribute, so the traceback shows both exceptions, making it clear the new one was triggered while handling the old one.

This is useful in layered systems — e.g., catching a low-level ValueError while parsing config and re-raising a higher-level ConfigError, without losing the original diagnostic information.

19 What's the difference between except Exception and a bare except:?

Bare except: also catches BaseException subclasses like KeyboardInterrupt.

A bare `except:` catches literally everything, including SystemExit and KeyboardInterrupt, which is almost always undesirable. `except Exception:` catches normal runtime errors while still letting those system-level signals propagate.

Best practice strongly favors `except Exception` (or even more specific types) over a bare except, since bare except can hide bugs, swallow Ctrl+C, and make debugging much harder.

20 How do you access the exception message/object in an except block?

Using `as e`, then str(e) or e.args.

Writing `except ValueError as e:` binds the exception instance to `e`. You can then print it directly (str(e) gives the message) or inspect `e.args` for the raw arguments it was created with.

Custom exceptions can also expose additional attributes (e.g., e.balance, e.order_id) set in their __init__, giving structured access to error context beyond just the message string.

21 What is the difference between IndexError and KeyError?

IndexError is for sequences (list/tuple), KeyError is for dictionaries.

IndexError occurs when you access a position outside a sequence's valid range (e.g., list[10] on a 3-item list). KeyError occurs when a dictionary lookup uses a key that doesn't exist.

Both are subclasses of LookupError. They're commonly avoided using `.get()` with a default for dicts, or by checking `len()`/`in` before indexing sequences.

22 What is the difference between ValueError and TypeError?

ValueError = right type, wrong value. TypeError = wrong type entirely.

ValueError occurs when a value has the correct type but an inappropriate value — like passing a non-numeric string to int(). TypeError occurs when an operation gets an object of the wrong type altogether, like adding a string to an integer.

Distinguishing these correctly in your own custom validation code is often asked in interviews as a sign of precise error-handling design.

23 Can you catch multiple exception types in a single except block?

Yes, using a tuple: except (TypeError, ValueError):

Python lets you group related exception types into a tuple in one except clause when they should be handled identically, avoiding repetitive code.

You can still bind the caught exception with `as e` even with a tuple, e.g., `except (TypeError, ValueError) as e:`, and inspect `type(e)` inside if you need to react differently per actual type.

24 What is a traceback?

A report showing the call stack at the point an exception occurred.

A traceback lists the sequence of function calls active when an exception was raised, from the outermost caller down to the exact line that failed, helping developers pinpoint the source of the error.

Tracebacks are attached to exception objects automatically and are printed when an exception goes uncaught; they can also be inspected programmatically via the `traceback` module for logging purposes.

25 What is the difference between ImportError and ModuleNotFoundError?

ModuleNotFoundError is a more specific subclass of ImportError.

ImportError is the general failure of an import statement. ModuleNotFoundError (added in Python 3.6) is raised specifically when the module itself can't be located, whereas plain ImportError can also occur when the module exists but a specific name inside it is missing.

Catching `ImportError` will also catch `ModuleNotFoundError` since it's a subclass, so code that needs to handle both generically can just catch the parent type.

26 Why should except blocks be ordered from specific to general?

Because Python stops at the first matching except block.

Python checks except clauses top-to-bottom and executes only the first one that matches. If a general exception type like Exception is listed before a specific one like ValueError, the specific block becomes unreachable dead code.

This is a genuine bug pattern interviewers test for — writing broad handlers first silently hides more specific, more informative error handling that will never execute.

27 What is the purpose of context managers (the "with" statement) in relation to exceptions?

They guarantee cleanup (like closing a file) even if an exception occurs.

The `with` statement wraps resource management so that a defined `__exit__` method always runs, closing files, releasing locks, or disconnecting from resources, whether the code inside succeeded or raised an exception.

This is functionally similar to using try/finally manually, but is more concise and less error-prone, and is Python's idiomatic solution for exception-safe resource handling.

28 What is the difference between logging an exception and re-raising it?

Logging records the error; re-raising still propagates it further up.

You can catch an exception, log it for diagnostics, and then use a bare `raise` to let it continue propagating so higher-level code (or the program itself) still knows the operation failed.

This pattern is common in layered applications: a lower layer logs technical details, then re-raises so an upper layer (e.g., an API handler) can convert it into a user-facing response, without losing either the log or the original traceback.

29 **What's the difference between `raise` and `raise e` inside an except block?**

Bare raise preserves the full original traceback; raise e can truncate it.

A bare `raise` re-raises the exact exception currently being handled, keeping its original traceback intact. Writing `raise e` re-raises the same object but can reset where the traceback appears to start, making debugging slightly harder.

For this reason, the bare `raise` form is the preferred idiom whenever you want to simply propagate the exception you just caught, unchanged.

30 **Can except blocks themselves raise new exceptions?**

Yes — and the new exception will show it occurred during handling of the first.

Code inside an except block can raise a completely different exception. Python automatically records this as "during handling of the above exception, another exception occurred," chaining them implicitly in the traceback.

This implicit chaining happens even without an explicit `from` clause, though using `raise NewError(...) from original` is preferred when you want to make the causal relationship explicit and intentional.

31 **Why shouldn't you use exceptions for normal control flow?**

Because it hurts readability and performance, and hides real errors among expected ones.

While Python does use exceptions for some normal flow (like StopIteration in iterators), overusing try/except for routine logic — e.g., using KeyError instead of checking `if key in dict` — makes code harder to read and can mask genuine bugs among expected "non-errors."

The general guidance is: use exceptions for genuinely exceptional, unexpected situations, and use simple conditionals for expected, routine branching whenever a clean check is available and doesn't cost much extra work.

32 **What is the EAFP principle and how does it relate to exceptions?**

"Easier to Ask Forgiveness than Permission" — try the operation, catch failure.

EAFP is a Python idiom where you attempt an operation directly inside a try block and handle the exception if it fails, rather than checking preconditions first (LBYL — "Look Before You Leap"). It's often preferred in Python for concurrency-safety and readability.

For example, opening a file directly and catching FileNotFoundError is more robust than checking `os.path.exists()` first, since the file's existence could change between the check and the actual open (a race condition).

33 **What is the difference between OSError and its subclasses like FileNotFoundError?**

OSError is the general parent for system-related errors; FileNotFoundError is one specific case.

OSError covers a broad range of operating-system level failures — permissions, disk issues, missing files. FileNotFoundError, PermissionError, and others are specific subclasses that let you react differently to each distinct situation.

In older Python 2 code you may see IOError, OSError, and WindowsError as separate classes — Python 3 unified these all under OSError, keeping the specific subclasses for finer-grained handling.

34 **Can you define a custom exception without any constructor?**

Yes — simply `class MyError(Exception): pass`.

If your custom exception doesn't need to carry extra data beyond a message, you don't need to override __init__ at all — it inherits Exception's default constructor, which just stores whatever arguments you pass to `args`.

You'd only add a custom __init__ when you need to attach structured extra information (like an error code or the invalid value) beyond the plain message string.

35 **What happens if you forget to call super().__init__() in a custom exception?**

str(exception) may not return your intended message properly.

If a custom exception's __init__ sets instance attributes but never calls `super().__init__(message)`, the exception's `args` won't be populated correctly, so printing it or logging it may show nothing useful instead of your message.

This is a common bug: always call `super().__init__(message)` (or pass all relevant args) so the exception behaves correctly with str(), repr(), and any tooling that relies on the standard Exception machinery.

36 **Is it possible to catch an exception raised in a different thread?**

Not directly with a normal try/except in the main thread.

An exception raised inside a separate thread doesn't propagate to the thread that started it. You typically need to capture it manually (e.g., store it in a shared variable, or use concurrent.futures, whose Future objects DO let you retrieve exceptions from worker threads).

This is why higher-level concurrency tools like `concurrent.futures.ThreadPoolExecutor` are preferred over raw `threading.Thread` when you need proper exception propagation and handling.

37 **What is the purpose of the `args` attribute on an exception?**

Stores the arguments passed when the exception was created.

Every exception instance has an `args` tuple containing whatever positional arguments were passed to its constructor — typically just the message string, but potentially more for custom exceptions.

`str(exception)` typically returns a formatted version of args. Custom exceptions overriding __init__ should ensure meaningful data ends up in args (via super().__init__) so tools relying on it work correctly.

38 **Can a try block have no except and only a finally?**

Yes — try/finally without except is valid.

Python allows a try block paired only with finally, with no except at all. In this case, any exception isn't handled or suppressed — it still propagates upward — but finally still runs its cleanup code first.

This pattern is common when you just need guaranteed cleanup (like closing a resource) without wanting to actually swallow or handle the error yourself.

39 **What's the risk of catching Exception too broadly in production code?**

It can hide bugs and make debugging much harder.

Catching Exception broadly (or worse, using a bare except) can silently swallow unexpected errors — typos, logic bugs, missing attributes — making them invisible instead of surfacing where they actually need fixing.

Best practice: catch the most specific exception types you actually expect and can meaningfully recover from; log broad catches loudly (with full traceback) rather than silently ignoring them, and avoid using broad catches as a substitute for fixing root causes.

40 **How do custom exceptions help with API error responses?**

They let a web framework map specific error types to specific HTTP status codes.

By defining exceptions like NotFoundError or ValidationError, a web framework's error handler can catch each type and return the appropriate HTTP status (404, 400, etc.) along with a structured error message, rather than a generic 500.

This is a very common real-world use case: custom exception hierarchies map cleanly onto API response contracts, keeping business logic free of HTTP-specific concerns while still enabling precise error responses.

41 **What is the difference between handling an exception and suppressing it?**

Handling responds meaningfully; suppressing just makes it disappear silently.

Handling an exception means doing something useful — logging it, retrying, returning a fallback value, informing the user. Suppressing means catching it and doing nothing (an empty except block), which hides real problems.

Interviewers often flag empty `except: pass` blocks as an anti-pattern; at minimum, log the exception even if you choose to continue execution after it.

42 **What module helps you work with tracebacks programmatically?**

The built-in `traceback` module.

The `traceback` module lets you format, print, or extract stack trace information programmatically — useful for structured logging systems that need to record full error context, not just the message.

Common functions include `traceback.format_exc()` to get the traceback as a string, and `traceback.print_exc()` to print it directly — both frequently used inside except blocks for logging.

43 **Can an exception class have multiple parent classes (multiple inheritance)?**

Yes, Python supports multiple inheritance for exception classes too.

Just like any Python class, a custom exception can inherit from more than one base exception class, letting it be caught by either parent's except clause — though this is used sparingly to avoid confusing hierarchies.

A practical use is combining a domain-specific base (like AppError) with a built-in category (like ValueError) so the exception is catchable both by application-wide handlers and generic ValueError handlers elsewhere.

44 **What's the difference between an exception's __str__ and __repr__?**

__str__ gives a user-friendly message; __repr__ gives a developer-friendly, unambiguous representation.

Calling `str(exception)` typically returns just the message text, while `repr(exception)` shows the exception's class name and its constructor arguments, useful for debugging and logs that need to distinguish exception types clearly.

Custom exceptions inherit sensible defaults for both from Exception, but can override __str__ or __repr__ if a more specific display format is needed.

45 **Why is precise exception handling considered good software engineering practice?**

It makes failures predictable, debuggable, and recoverable instead of chaotic.

Precise, well-thought-out exception handling turns unpredictable failures into predictable, well-documented behavior — the application knows exactly what can go wrong, responds appropriately, and gives developers clear diagnostic information when things do fail.

This directly impacts reliability, maintainability, and user trust: applications that fail gracefully with clear error messages and safe fallbacks are dramatically easier to operate, debug, and extend than ones riddled with unguarded assumptions.

11. Quick Revision

11.1 Cheat Sheet — Syntax at a Glance

```
try:
    risky_code()
except SpecificError as e:
    handle(e)
except (TypeA, TypeB) as e:
    handle_multiple(e)
else:
    only_if_no_exception()
finally:
    always_runs()

raise ValueError("message")          # trigger an exception
raise                                  # re-raise current exception (inside except)
raise NewError("msg") from original    # chain exceptions

assert condition, "message"           # debug-only check -> AssertionError

class MyError(Exception):
    def __init__(self, msg, extra=None):
        super().__init__(msg)
        self.extra = extra
```

11.2 Comparison Tables

Exception	Cause
ValueError	Right type, bad value
TypeError	Wrong/incompatible type
IndexError	Sequence index out of range
KeyError	Dict key missing
NameError	Undefined variable/name
ZeroDivisionError	Division/modulo by zero
FileNotFoundError	File/path missing
AttributeError	Missing attribute/method
ImportError / ModuleNotFoundError	Import failed / module missing

Python	Java
try / except	try / catch
raise	throw
BaseException	Throwable
No checked exceptions	Checked + unchecked exceptions
else clause available	No else clause

raise	assert
Always executes	Can be disabled with -O flag
Any exception type	Always AssertionError
Safe for production validation	Debugging/dev use only

11.3 Common Mistakes to Avoid

- Using a bare `except:` instead of `except Exception:` — accidentally catches KeyboardInterrupt/SystemExit.
- Putting a general exception (e.g., `Exception`) before more specific ones — makes the specific except blocks unreachable.
- Using `assert` to validate user input or security-sensitive conditions — it can be compiled away with `-O`.
- Forgetting `super().__init__(message)` in a custom exception's constructor — breaks `str(exception)`.
- Silently swallowing exceptions with an empty `except: pass` — hides real bugs.
- Relying on `finally` with a `return` statement — can silently discard a pending exception.

- Checking file existence before opening it (LBYL) instead of just trying and catching `FileNotFoundError` (EAFP) — introduces race conditions.
- Confusing `raise e` with a bare `raise` when re-raising — the bare form preserves the full original traceback.

11.4 Last-Minute Interview Tips

- Be ready to explain the execution order of try/except/else/finally with a simple diagram — this is asked in almost every interview.
- Know the exception hierarchy at least three levels deep (BaseException → Exception → ArithmeticError/LookupError/etc.) — it signals real understanding, not memorized syntax.
- Practice writing a custom exception with a constructor that stores extra context (like balance or error_code) — this is a very common live-coding ask.
- Always mention **why** you'd choose a specific exception type over a generic one — interviewers value reasoning over syntax recall.
- If asked to compare Python and Java, lead with the checked-vs-unchecked distinction — it's the single most "insightful" point you can make.
- When explaining assert vs raise, always mention the `-O` flag — it's the detail that separates a strong answer from a surface-level one.
- Practice explaining EAFP vs LBYL with the file-existence example — it's a favorite "Pythonic thinking" question.

> **Final tip:** In interviews, always tie an exception-handling answer back to *why* it matters for real software — reliability, debuggability, and safe resource cleanup — not just the syntax itself. That's what distinguishes a strong candidate from someone who has only memorized keywords.